

技术文档

System Design Document

Version 1.0

June 10, 2026

AgentHub

多 Agent 协作平台

小组: 启灵

齐紫瀚 熊子恒 张桐铖

赛事: AI 全栈挑战赛——AgentHub 多 Agent 协作平台

Submitted in partial fulfillment of the requirements

覆盖: 系统分层、部署拓扑、数据模型 (完整 DDL)、编排内核 (状态机 / 调度 / 重试 / 崩溃恢复 / 不变量)、双层 Planner、统一适配器层 (CLI 子进程 + NDJSON 事件映射)、凭证解析链、事件投影与实时性、共享工作区与安全、HTTP API 参考、三条核心链路时序、测试门禁、设计 vs 实现对照、已知局限。

全文描述均以仓库代码为准并标注文件路径 (internal/agenthub/**)。凡未实现者明确标注 **路线图**, 不混入”已实现”叙述。

Contents

1	系统总览	3
1.1	分层架构	3
1.2	部署拓扑 (开发环境)	3
1.3	关键选型与理由	4
2	数据模型 (完整 DDL)	4
2.1	事件类型全表 (orchestrator/types.go)	7
2.2	设计要点	7
3	编排内核: 状态机与调度	8
3.1	Run 生命周期 (state_machine.go canTransitionRun)	8
3.2	Task 生命周期 (state_machine.go canTransitionTask)	8
3.3	Reconcile 驱动模型 (service.go ReconcileRun)	8
3.4	关键不变量	9
4	双层 Planner	9
4.1	决策顺序	10
4.2	LLM 输出的防御性解析 (llm_planner.go parsePlannerOutput)	10
4.3	上下文注入	11
5	统一适配器层	11
5.1	接口契约 (orchestrator/interfaces.go)	11
5.2	CLI 型: 公共骨架 + 两个回调 (clirunner.go)	11
5.3	框架型 (Memoh): 接口反转 (providers/memoh.go)	12
5.4	接入一个新平台的成本	12
6	凭证解析链 (单聊 / 编排同源)	12
7	事件投影: 运行事实 → 聊天叙事 (orchestrator_projector.go)	13
8	实时性设计	13
9	共享工作区与安全	14
10	HTTP API 参考	14
11	三条核心链路时序	14
12	测试与质量门禁	15
13	设计 vs 实现对照 (把产品愿景映射到代码, 诚实标注)	16
14	已知局限与演进方向 (诚实清单)	16

1 系统总览

1.1 分层架构

系统自上而下分为六层。依赖方向严格单向、杜绝包环:handlers → agenthub(宿主) → orchestrator(领域) ← providers。领域包 orchestrator 不 import 任何业务包,可独立单测、可替换宿主。



Figure 1: 分层组件图: 六层结构与单向依赖。

1.2 部署拓扑 (开发环境)

mise run dev 起 docker compose(devenv/docker-compose.sqlite.yml) 四容器:web(Vite 热更新,:19082)、server(Go 后端 + air 热重载, 仓库挂载为 /workspace)、sparse(稀疏检索)、qdrant(向量库)。

进程拓扑要点:

- **Memoh bot** 各自再起嵌套 containerd 容器 (workspace-<botID>) 作为工具沙箱,UDS-gRPC 通信;

- **Claude Code / Codex CLI** 作为 server 容器内子进程运行, 工作目录为容器内 `os.TempDir()/memoh_agenthub_rooms/<roomID>`(房间共享目录, 见 §9);
- 编排内核与 HTTP 服务同进程; 无独立调度 worker(见 §3.3 reconcile 驱动模型)。

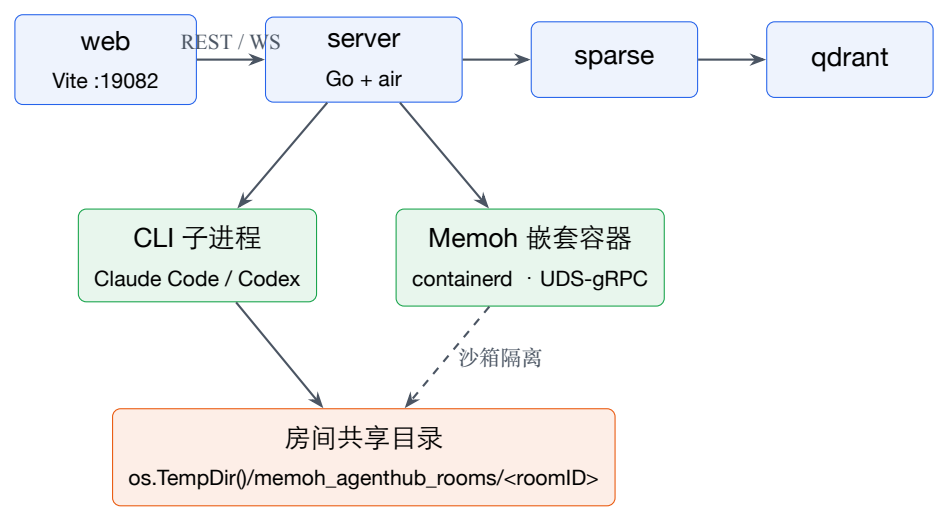


Figure 2: 部署图: 四容器 + CLI 子进程 + Memoh 嵌套沙箱 + 房间共享目录。

1.3 关键选型与理由

选型	理由
uber/fx 依赖注入	Provider / Store / Resolver 声明式装配; 新增 Agent 平台 = 注册一个 Provider, 不动调用方 (orchestrator_service.go NewOrchestratorService)
SQLite / Postgres 同构	同一 SQLStore 用 dialect 分支跑两库; 开发零依赖 (SQLite), 生产可换 Postgres,DDL 在 sql_store.go 内联建表
编排做成纯领域包	orchestrator 不 import 业务层 → 可单测、可换宿主;Memoh 接入用接口反转避免 flow→botruntime→providers 包环
CLI 子进程而非 SDK	Claude Code / Codex 的真实能力在 CLI(工具、文件操作、多轮 agentic loop); 封装其 stream-json 输出比重写 agent loop 既便宜又保真
reconcile 轮询而非常驻 worker	简化崩溃恢复与幂等 (§3.3); 实时性由增量事件接口补足 (§8)
事件溯源 (event-sourcing lite)	agent_hub_run_events 是唯一事实源;UI 投影、实时气泡、审计、回放全部从它派生

2 数据模型 (完整 DDL)

两组表。IM 域 (房间 / 成员 / 消息) 与编排域 (run / task / dep / attempt / event)。以下为编排域的真实建表语句 (orchestrator/sql_store.go,SQLite 方言;Postgres 同构, 类型映射 TEXT↔UUID、INTEGER↔BIGINT、TEXT↔JSONB):

```
-- 一次用户触发的编排生命周期
CREATE TABLE agent_hub_runs (
```

```

id TEXT PRIMARY KEY,
room_id TEXT NOT NULL REFERENCES agent_hub_rooms(id) ON DELETE CASCADE,
trigger_message_id TEXT NOT NULL DEFAULT '',
objective TEXT NOT NULL,
status TEXT NOT NULL,           -- draft/planning/dispatching/collecting/
                                  completed/failed/cancelled
planner_version TEXT NOT NULL DEFAULT '',
created_by TEXT NOT NULL DEFAULT '',
metadata TEXT NOT NULL DEFAULT '{}', -- 含 room_history(置顶 + 近 20 条)
created_at_ms INTEGER NOT NULL,
updated_at_ms INTEGER NOT NULL
);
CREATE INDEX idx_agent_hub_runs_room_updated ON agent_hub_runs(room_id, updated_at_ms
DESC);

-- DAG 节点
CREATE TABLE agent_hub_tasks (
id TEXT PRIMARY KEY,
run_id TEXT NOT NULL REFERENCES agent_hub_runs(id) ON DELETE CASCADE,
parent_task_id TEXT,           -- 预留给多级编排,当前未用于分层
title TEXT NOT NULL,
description TEXT NOT NULL,
assigned_agent_id TEXT NOT NULL DEFAULT '',
provider_name TEXT NOT NULL DEFAULT '', -- claudecode/codex/memoh/noop
priority INTEGER NOT NULL DEFAULT 0,
status TEXT NOT NULL,         -- pending/ready/running/succeeded/failed/
                                blocked/cancelled
timeout_ms INTEGER NOT NULL DEFAULT 0,
max_retries INTEGER NOT NULL DEFAULT 0,
attempt_count INTEGER NOT NULL DEFAULT 0,
idempotency_key TEXT NOT NULL DEFAULT '',
metadata TEXT NOT NULL DEFAULT '{}',
created_at_ms INTEGER NOT NULL,
updated_at_ms INTEGER NOT NULL
);
CREATE INDEX idx_agent_hub_tasks_run_priority
ON agent_hub_tasks(run_id, status, priority DESC, created_at_ms ASC);

-- 显式 DAG 边
CREATE TABLE agent_hub_task_deps (
run_id TEXT NOT NULL REFERENCES agent_hub_runs(id) ON DELETE CASCADE,
task_id TEXT NOT NULL REFERENCES agent_hub_tasks(id) ON DELETE CASCADE,
depends_on_task_id TEXT NOT NULL REFERENCES agent_hub_tasks(id) ON DELETE CASCADE,
PRIMARY KEY (task_id, depends_on_task_id)
);

-- 每次执行记录(重试不覆盖历史)
CREATE TABLE agent_hub_task_attempts (

```

```

id TEXT PRIMARY KEY,
run_id TEXT NOT NULL REFERENCES agent_hub_runs(id) ON DELETE CASCADE,
task_id TEXT NOT NULL REFERENCES agent_hub_tasks(id) ON DELETE CASCADE,
attempt_no INTEGER NOT NULL,
provider_name TEXT NOT NULL,
agent_id TEXT NOT NULL DEFAULT '',
status TEXT NOT NULL,          -- running/succeeded/failed/timed_out/
                                cancelled
input_payload TEXT NOT NULL DEFAULT '{}',
output_payload TEXT NOT NULL DEFAULT '{}',
error_message TEXT NOT NULL DEFAULT '',
retryable INTEGER NOT NULL DEFAULT 0,
started_at_ms INTEGER NOT NULL,
finished_at_ms INTEGER,
idempotency_key TEXT NOT NULL,
UNIQUE (task_id, attempt_no)    -- 防同一尝试重复落库
);

-- append-only 事件流:投影/实时/审计/回放的唯一事实源
CREATE TABLE agent_hub_run_events (
  id TEXT PRIMARY KEY,
  run_id TEXT NOT NULL REFERENCES agent_hub_runs(id) ON DELETE CASCADE,
  task_id TEXT,
  seq INTEGER NOT NULL,
  type TEXT NOT NULL,
  payload TEXT NOT NULL DEFAULT '{}',
  created_at_ms INTEGER NOT NULL,
  UNIQUE (run_id, seq)         -- seq 在 run 内单调
);
CREATE INDEX idx_agent_hub_run_events_seq ON agent_hub_run_events(run_id, seq ASC);

```

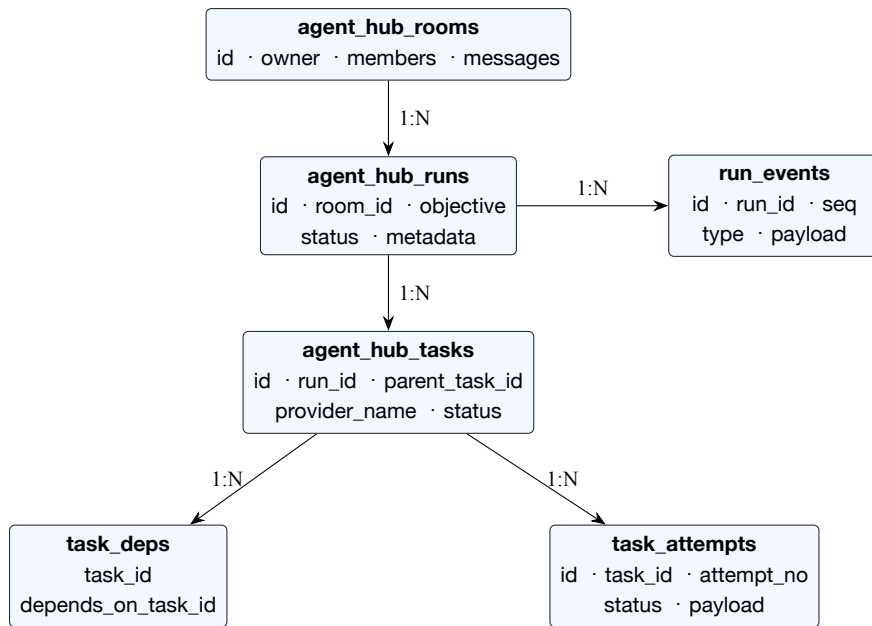


Figure 3: ER 图: 编排域五表与 IM 域 rooms 的主外键关系 (均为 1:N, 删除级联)。

2.1 事件类型全表 (orchestrator/types.go)

事件	触发点	是否投影到聊天
run_created	StartRun 建 run	否
run_planned	tasks 落库后	是 (任务规划 + 清单)
run_status_changed	任何 run 状态转移	仅 completed / failed
task_created	CreateTasks	否
task_ready	依赖求解通过	否
task_dispatched	派发给 Provider	是 (开始执行)
task_succeeded	attempt 成功	是 (产出正文, 去重)
task_failed / task_timed_out	attempt 失败 / 超时且终态	仅终态失败 (降级)
task_blocked	上游失败连锁阻塞	否
task_retried	failed→ready 回炉	否
task_cancelled	CancelRun	否
agent_tool_call	CLI 子进程发起工具调用	否 (进 R9 过程气泡)
agent_output	CLI / Memoh 产出文本	是 (text/result/error;thinking/init 过滤)

2.2 设计要点

- 事件表是事实源:seq 在 run 内单调,UI 投影、实时气泡、审计、回放全部从它派生;
- **attempts** 与 **tasks** 分离: 重试不覆盖历史, 每次执行的输入 / 输出 / 错误都可追溯;
- 幂等键:idempotencyKey = sha256(runID|taskID|attemptNo|description)(service.go idempotencyKey), 防重复派发;

- 每条投影消息携带 `{run_id, event_seq, event_type}`: 为”投影幂等下沉到插入层”预留唯一键 (见 §14 局限 (1))。

3 编排内核: 状态机与调度

3.1 Run 生命周期 (`state_machine.go canTransitionRun`)

`dispatching` $\rightleftharpoons$ `collecting` 的往返即 DAG 推进: 有任务在跑 $\rightarrow$ `collecting`; 一批完成解锁下游、还有待派 $\rightarrow$ 回 `dispatching`。

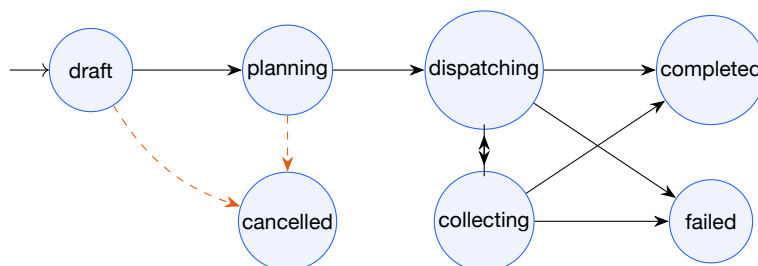


Figure 4: Run 生命周期状态机 (任意非终态可转 cancelled; 终态不可再转移)。

3.2 Task 生命周期 (`state_machine.go canTransitionTask`)

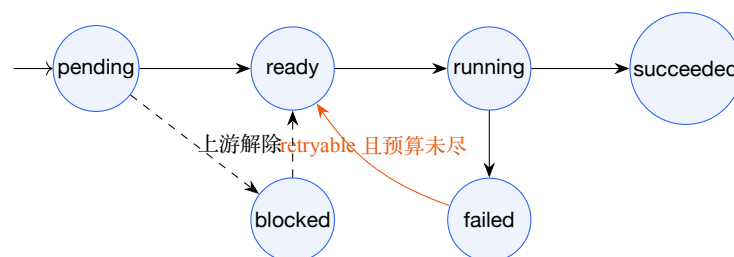


Figure 5: Task 生命周期状态机 (failed 在重试预算内回 ready; 上游失败致 blocked)。

- `markRunnableTasks(service.go)`: 依赖全部 succeeded $\rightarrow$ ready; 任一上游 failed/blocked/cancelled $\rightarrow$ blocked。连锁阻塞只阻塞下游, 不阻塞旁路分支——这是失败降级的机制本体;
- 重试语义: `MaxRetries` 是额外尝试数。`AttemptCount` 在派发前自增 (故首跑 == 1), `AttemptCount` $\leq$ `MaxRetries` 时 failed $\rightarrow$ ready 回炉, 发 `task_retried`;
- run 终态判定 `recomputeRunStatus`: 全 succeeded $\rightarrow$ completed; 存在 failed/blocked 且无可推进 $\rightarrow$ failed; 有 running $\rightarrow$ collecting; 否则 $\rightarrow$ dispatching。

3.3 Reconcile 驱动模型 (`service.go ReconcileRun`)

没有常驻调度线程。一切推进都由幂等入口 `ReconcileRun(runID)` 完成:

```

ReconcileRun:
  loop(<=100 次):
    failTimedOutTasks    # 超时判定 + 重启自愈(见下)
    markRunnableTasks    # 依赖求解 pending/blocked -> ready
  
```

```
recomputeRunStatus    # run 状态推进;终态则退出
dispatchReadyTasks    # 按约束派发;DispatchAsync 下 go 出去后 break
(sync 模式重读 run 继续循环;async 模式 break,由 goroutine 完成回调再次 reconcile)
```

调用来源三处,任意一处到达都能把 DAG 推到底——谁触发不重要、幂等才重要:

1. `StartRun(AutoDispatch=true);`
2. 异步任务 goroutine 完成后的自回调 (`context.WithoutCancel` 脱离 HTTP 生命周期);
3. 前端 1.5s 轮询 `POST /runs/:id/reconcile`。

生产装配 (`orchestrator_service.go`): `MaxParallelPerRun=3`(单 run 总并发)、`MaxParallelPerAgent=1` (同 Agent 串行——CLI Agent 共享工作目录,并发写同目录有害)、`DispatchAsync=true`、`DefaultTaskTimeout=10min`。

调度约束 (`dispatchReadyTasks`): ready 队列按 priority 降序 + 创建时间; `provider_name` 解析失败回退到注册表 fallback; 超并发上限则本轮不派。

崩溃恢复: 任务 goroutine 随进程死亡。 `failTimedOutTasks` 比较 `attempt.StartedAt` 与进程启动时间 `s.startedAt`——发现”开始于上一个进程生命周期”的 running attempt, 立即判 `task executor was interrupted by server restart` 并按 retryable 处理。重启后第一次 **reconcile** 即自愈, 无需额外恢复协议。

3.4 关键不变量

1. 进入引擎的每个 Plan 必过 `validatePlan`(key 非空唯一、依赖指向真实 key);
2. 同一 (`task_id`, `attempt_no`) 至多一条 attempt (DB UNIQUE);
3. (`run_id`, `seq`) 单调唯一 (DB UNIQUE);
4. 任一 run 在有限次 reconcile 内收敛到终态 (循环上限 100 + 无可派发即 break)。

4 双层 Planner

4.1 决策顺序

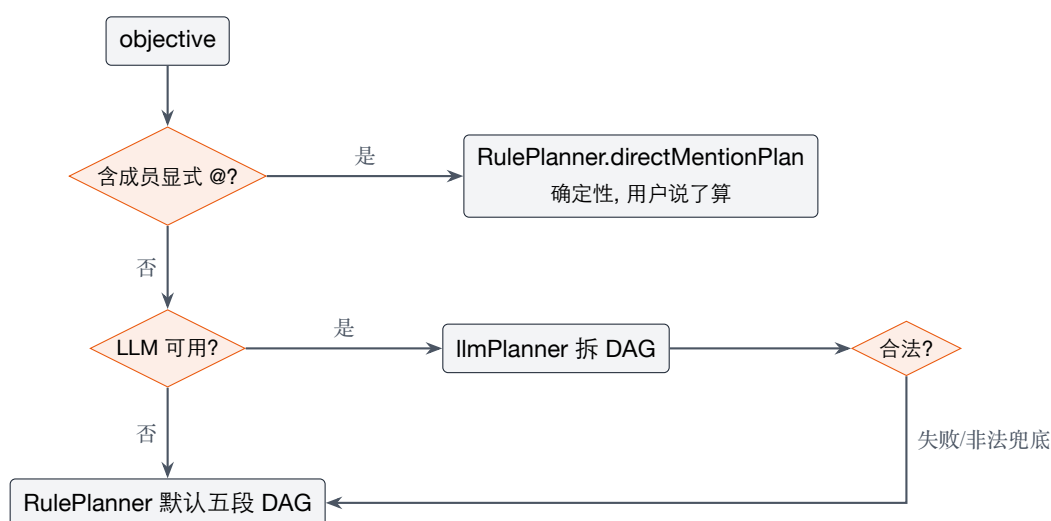


Figure 6: 双层 Planner 决策流程: 显式 @ → 确定性规则; 否则 LLM, 失败回落规则。

- **directMentionPlan(planner.go)**: 按 @ 在文中出现位置排序被点名成员, 逐个生成任务并以前一个为依赖 (顺序流水线)。mention 匹配带边界检查 (isMentionChar: 后随字符不是 [a-z0-9-_.:/@] 才算命中, 防 @cc 误中 @ccc), 维护别名集 (显示名 / bot UUID / provider 名 / cc · ds · cx 缩写, 见 agentMentionAliases);
- 默认五段 **DAG(planner.go Plan):** plan → backend || frontend → verify → finalize, Agent 按能力关键词匹配 (pickAgent)。这是无 LLM 时的保底结构。注意: 此处 verify 仅是一个按关键词派给某 Agent 的普通任务, 非独立的信息不对称校验;
- **LLM Planner(llm_planner.go)**: 温度 0.2、45s 超时, system prompt 约束”严格只输出任务 DAG 的 JSON”(全文见 plannerSystemPrompt), user prompt 携带群聊近期对话 + 可用 Agent 的 id / 名称 / 能力清单。

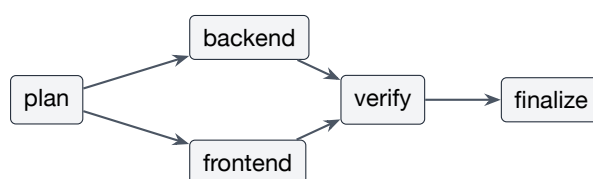


Figure 7: 默认五段 DAG: plan → backend || frontend → verify → finalize。

4.2 LLM 输出的防御性解析 (llm_planner.go parsePlannerOutput)

LLM 输出不可信, 逐层设防:

1. 容忍前导散文与 ```json 围栏, 抽首个 {...}(extractJSONObject);
2. 任务 key 去重补缺;
3. agent_id 不在房间成员表 → 强制改派默认成员 (LLM 幻觉出的 Agent 不会进入调度);
4. 悬挂 / 自指依赖剪除;
5. Kahn 算法检测依赖环 (hasDependencyCycle), 成环即整体拒绝回落规则 Planner。

最终保证: 进入引擎的 Plan 一定通过 validatePlan,Planner 层的任何失败都不会让 run 起不来。

4.3 上下文注入

StartRun(orchestrator_service.go) 把两段上下文写进 run metadata 的 room_history:

- 置顶长期上下文 (从 room.metadata.pinned_message_ids 反查消息原文, 标注「务必优先考虑」);
- 近 20 条滚动对话 (recentRoomHistory,oldest→newest)。

Planner 用它理解意图; 每个子任务执行时由 PromptWithContext(providers/prompt.go) 再拼进 CLI prompt, 并标注” 仅供理解上下文, 请聚焦当前任务”。

5 统一适配器层

5.1 接口契约 (orchestrator/interfaces.go)

```
type AgentProvider interface {
    Name() string // 注册名:claudecode / codex / memoh / noop
    Capabilities() []string // 参与 Planner 能力匹配
    Execute(ctx, ExecuteTaskRequest) (ExecuteTaskResult, error)
}
```

ExecuteTaskRequest 带 Run(含 room_history metadata)、Task、AttemptNo、Upstream(上游成功 attempt 的产出, 供下游消费)。ExecuteTaskResult.Retryable 由 Provider 给出错误分类 (限流 / 超时 → 可重试; 鉴权失败 → 不可重试), 引擎据此走 §3.2 重试或降级。

5.2 CLI 型: 公共骨架 + 两个回调 (clirunner.go)

CLIRunner.Run 承担所有 CLI 共性:

1. **fail-fast** 二进制探测 (LookPath, 缺失即 ErrCLINotFound 不可重试);
2. 启动子进程 (2h 硬超时,ExecRequest.Timeout);
3. 跨 chunk 的行缓冲 NDJSON 解析:stdout 可能在任意字节处切分,lineBuffer 拼回完整行后按 \n 切分; 不可解析的行优雅跳过;
4. 每行 ParseEvent → CLIEvent, 回调 OnEvent;
5. 文本聚合:text 事件累计;result 在文本提示路径累计 (stream-json 输入路径下跳过 result 防重复)。

单个平台只需提供两件事:

平台	BuildArgs(cli_helpers.go)	NDJSON 解析 (ParseEvent)
Claude Code	-p <prompt> --output-format stream-json --verbose --permission-mode auto --max-turns 15 [--model <m>]	ClaudeParseEvent:system/init、 assistant(thinking/text/tool_use)、 user(tool_result)、 result/error → 统一 CLIEvent
Codex	exec --json --skip-git-repo-check --sandbox workspace-write [--model <m>] <prompt>	CodexParseEvent:thread.started / turn.started、 item.completed(message/command)、 turn.completed、 error

统一中间事件 CLIEvent{Type: init|thinking|text|tool_use|tool_result|result|error|turn, Content, ToolName, Payload, SessionID, Raw}。OnEvent 把每个 CLIEvent 直接落库为 RunEvent(agent_output / agent_tool_call)——这条事件流就是 **R9** 实时气泡的数据源, 适配器层天然为实时性供数, 无需额外打点。

第三方端点鉴权 (ClaudeEnv):base_url 非 api.anthropic.com 时改用 ANTHROPIC_AUTH_TOKEN(Bearer) 并显式清空 ANTHROPIC_API_KEY(x-api-key 头)——这是 DeepSeek 等 Anthropic 兼容端点能跑通的关键细节。日志中所有 key / token 仅打印末 4 位预览 (clirunner.go safeEnv)。

5.3 框架型 (Memoh): 接口反转 (providers/memoh.go)

自建 bot 跑在 memoh 框架 (internal/conversation/flow), 但 agenthub 不能 import flow(包环 flow->botruntime->agenthub/providers)。解法:agenthub 侧只定义最小接口

```
type MemohRunner interface { RunTurn(ctx, RunTurnInput) (RunTurnResult, error) }
```

由组合根 cmd/agent 用 flow.Resolver.Chat 实现并经 fx 注入。会话隔离:SessionID 直接复用 run 的 UUID——每次编排一个独立会话, 同 run 内多个 memoh 任务共享多轮记忆, 且永不污染用户的 1:1 聊天历史。runner 缺失时显式报 ErrMemohRunnerMissing 而非静默空成功。

5.4 接入一个新平台的成本

CLI 型 ≈ 一个 BuildArgs + 一个 ParseEvent(约百行)+ 注册表一行;HTTP 型 ≈ 实现 Execute 直调 API。凭证沿用「通用设置」Provider 体系 (§6), 无新配置面。**Codex 适配器 (providers/codex.go)** 已按此骨架完成并注册, 处于”代码就绪、未在演示中启用”状态 (启用属配置 / 验证工作, 非开发)。

6 凭证解析链 (单聊 / 编排同源)

cc 任务执行时的配置解析顺序 (orchestrator_service.go claudeBotConfigResolver):

1. bot 自身 provider_ext["claudecode"] → api_key/auth_token/base_url/model/permission_mode
 2. 通用设置 anthropic 兼容 Provider → ResolveCredentialsForFramework("claudecode") 取 key+base_url
 3. 进程环境变量 → 兜底(provCfg.FromEnvWithDefaults)
- 最终 APIKey 与 AuthToken 均空 → 指路型中文错误(不可重试, run 走降级)

与单聊 botruntime/cli.go 的 mergeClaudeCodeConfig 完全同序——这是” 同一个 bot 单聊能跑、编排也必须能跑” 的工程保证。其中第 1 层携带 bot 的模型至关重要: 不带 --model 时 CLI 默认 Claude 系模型,DeepSeek 端点不识别 → 只回 system 事件零产出 (R7 修复的根因)。

LLM Planner 的模型解析 (resolvePlannerModel) 同理: 优先 openai-completions 型 Provider 再 anthropic-messages, 跳过 DeepSeek 不支持的 openai-responses。

7 事件投影: 运行事实 → 聊天叙事 (orchestrator_projector.go)

投影是纯函数 runEventsToMessages(events, run, tasks, after) -> ([]CreateMessageRequest, maxSeq), 无 IO, 可直接用真实引擎事件流做表驱动测试。规则:

事件	投影消息	备注
run_planned	「任务规划」+ 任务清单 (按创建序 / 优先级排序)	system 消息
task_dispatched	「开始执行:< 标题 >」	agent 状态消息
agent_output	Agent 产出消息	仅投 text/result/error;thinking/init/turn/tool 噪音被 agentOutputBody 滤掉
task_succeeded	最终产出 (output.raw_output)	与已投流式正文归一化去 重 (surfacedAgentOutput), 不重复刷屏
task_failed / task_timed_out	「失败 + 原因 +(已降级, 继 续执行其余任务)」	仅终态失败投影; 重试后成 功的中间失败不投
run_status_changed→completed/failed	「协作完成」/ 「以失败结 束」	

幂等: 进程内 projSeq[runID] 高水位 + ListEvents(afterSeq) 增量拉取; 每条投影消息 metadata 携带 {run_id, event_seq, event_type}。投影由宿主层在 StartRun / ReconcileRun 后同步调用 (OrchestratorService.projectRun)。

8 实时性设计

单聊 (R5):connectWebSocket(botId) → 发 {message, session_id} → 流式 UIMessage(text/reasoning/tool 按块 id 去重累积)→ onProgress 驱动「正在输入…」草稿气泡 → end 事件落正式消息。会话按 room+agent 持久复用, 多轮记忆在 bot 侧。

群聊编排 (R9): 双通道并行——

1. 轮询通道:1.5s POST /runs/:id/reconcile(顺带推进 DAG)+ 失效 latest-run / messages 查询缓存 → 任务面板与投影消息刷新;

2. 事件通道:GET /runs/:id/events?after_seq= 增量拉原始事件, 前端把投影器故意不投的过程信号 (thinking、tool_use) 折叠进每个 running 任务的临时气泡; 任务离开 running 即清除, 由正式消息接管。正文不进气泡——它已被投影, 重复渲染会污染时间线。

为什么不用 WS 推送:reconcile 本就需要外部周期驱动 (崩溃恢复语义), 轮询顺路带回事件, 零新增通道与实体; 事件模型 (seq 增量) 与传输无关, 未来换 WS 推送只动传输层。

9 共享工作区与安全

目录模型 (providers/workspace_resolver.go): 解析顺序 task.metadata.workspace_path(显式指定)→ os.TempDir()/memoh_agenthub_rooms/<roomID>(房间共享目录, 编排常态路径) → workspace.Manager(无房间时按 Agent)→ cwd。选 host 临时目录而非 bot 容器路径的原因:CLI 子进程跑在 server 进程同侧,bot 嵌套容器内的 /data 对它不可达; 同房间所有 CLI Agent 落同一目录,A 写的文件 B 直接可读,prompt 显式注明 (WithWorkdirNote)。

工作区 API 安全 (handlers/agent_hub_workspace.go): 三端点 (列文件 / 读文件 / 执行命令) 全部先走 JWT → service.Get(owner, room) 属主校验; 路径做穿越防护 (强制绝对化后校验前缀); 执行命令 30s 超时、输出 64KiB 截断、cwd 锁定房间目录; 文件列表上限 500、单文件读 256KiB,.git / node_modules / target 过滤。定位是房主单租户的开发工作台, 不承诺对抗恶意 Agent 的沙箱 (那是 bot 容器的职责)。

10 HTTP API 参考

编排端点 (handlers/agent_hub_orchestrator.go, 前缀按路由组):

方法	路径	用途
POST	/rooms/:room_id/runs	启动编排 run(objective + agents + auto_dispatch)
GET	/rooms/:room_id/runs/latest	房间最近一次 run 快照 (无则 404)
GET	/runs/:run_id	run 快照 (run + tasks + deps + attempts)
POST	/runs/:run_id/reconcile	推进 DAG(前端 1.5s 轮询)
POST	/runs/:run_id/cancel	取消 run(级联取消未终态任务)
GET	/runs/:run_id/events?after_seq=&limit=	增量拉原始事件 (R9 过程气泡)
POST	/runs/reconcile-active	批量推进活跃 run(当前无自动调用方, 见 §14 (1))

房间 / 消息 / 工作区端点见 agent_hub.go / agent_hub_workspace.go。所有端点先 JWT + 房间属主校验。

11 三条核心链路时序

链路 A · 群聊自动编排 (场景 B)

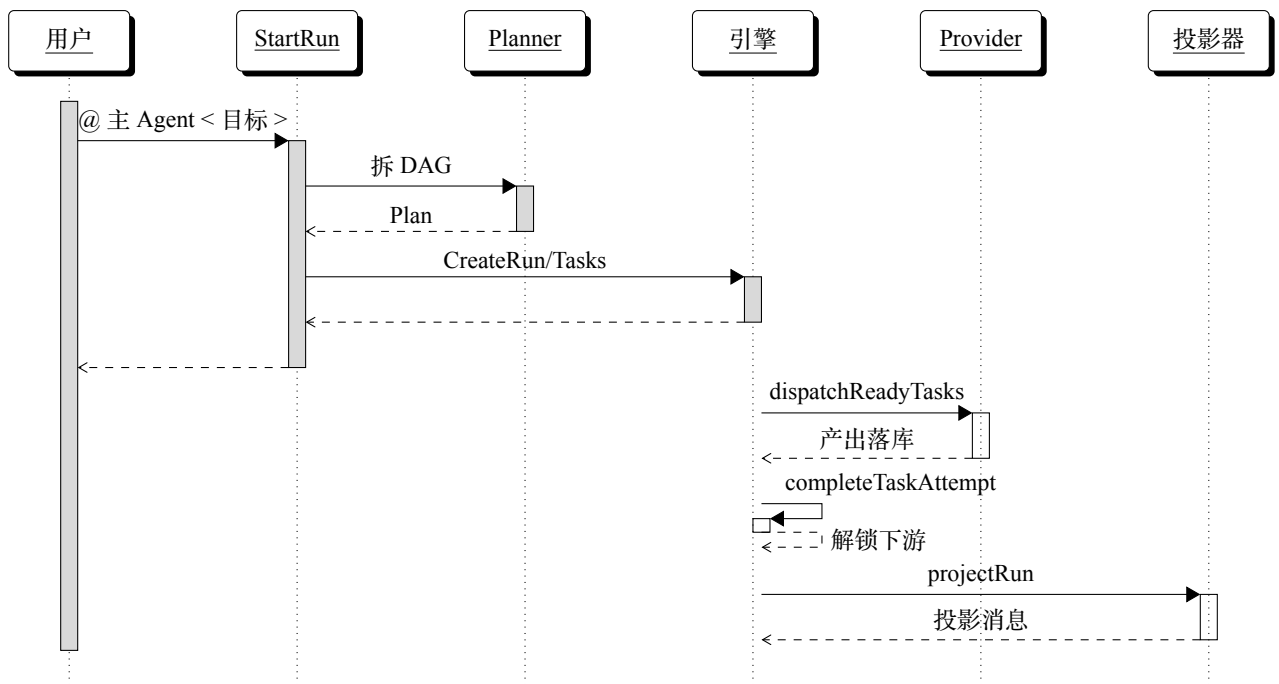


Figure 8: 链路 A 时序图: 群聊自动编排端到端 (前端另以 1.5s reconcile + after_seq 拉事件渲染过程气泡)。

链路 B · 显式 @ 流水线 (场景 C): StartRun → hasExplicitAgentMention 命中 → 委托 RulePlanner.directMentionPlan(按 @ 顺序生成顺序依赖链) → 其余同链路 A。

链路 C · 崩溃自愈: 进程重启 → 前端轮询 reconcile → failTimedOutTasks 发现 attempt.StartedAt < s.startedAt 的 running attempt → 判 interrupted、retryable → failed → ready 回炉 → 重新派发。

12 测试与质量门禁

- 编排引擎: 状态机 / 调度 / 重试 / 超时 / 取消在 orchestrator 包内单测 (service_test.go、latest_run_test.go);
- Planner: llm_planner_test.go(防御性解析、改派、剪依赖、检测环)、规则 Planner mention 匹配;
- 投影: orchestrator_projector_test.go 用真实引擎产生的事件序列断言纯函数 runEvents ToMessages;
- 适配器: claudecode_test.go / codex_test.go / clirunner_test.go(NDJSON 解析、跨 chunk 缓冲、参数构造);
- 端到端: Playwright 驱动真实浏览器验收 (单聊流式、群聊 run 实时气泡、工作区文件 / 终端), 见 01-demo-runbook.md;
- 门禁: husky pre-commit 强制 golangci-lint + go test ./... + eslint, 不绿不入库。

13 设计 vs 实现对照 (把产品愿景映射到代码, 诚实标注)

与产品设计文档 §2 / §6 配套。左列是产品 / 学术设计中提出的机制, 右列是代码现实。

设计机制	代码现实	状态
任务拆解 + 并行调度 + 聚合	LLM / 规则双层 Planner + DAG 引擎 + 投影	已实现
失败降级、重试预算、超时	completeTaskAttempt / failTimedOutTasks	已实现
崩溃自愈	进程启动时间比较	已实现
多轮上下文 / 置顶长期记忆	room_history(置顶 + 滚动) 注入	已实现
跨模型族 (规避 Model Monoculture)	ClaudeCode + Memoh(DeepSeek);Codex 就绪未启用	部分实现
显式 @ > LLM 决策	hasExplicitAgentMention → directMentionPlan	已实现
事件溯源 / 审计 / 回放	agent_hub_run_events(seq 单调)	已实现
Plan 优先交互 (确认闸)	run 自动派发, ”规划” 为执行后叙事	路线图
接口契约 (生成 / 传递 / 协商 / 验证)	无; 共享目录 + DAG 依赖近似	路线图
独立 Verification Agent(信息不对称校验)	verify 仅普通子任务	路线图
PRD 文档 + 版本控制 + 定向重对齐	无	路线图
多级 Sub-Orchestrator + 预评估路由	仅扁平 DAG;parent_task_id 预留未用	路线图
Markov 安全监控 / 编排侧活锁循环检测	仅 CLI --max-turns + 超时; 单 bot 侧 LoopGuard 未接入	路线图
每任务硬性 token 预算	无	路线图

14 已知局限与演进方向 (诚实清单)

1. 投影幂等的进程内边界:projSeq 高水位在内存, 进程重启后若有人手动调 POST /runs/reconcile-active 会重投历史消息。修复路径已写在代码注释 (orchestrator_service.go DURABILITY CONTRACT): 利用消息 metadata 已带的 (run_id, event_seq) 加唯一约束 + ON CONFLICT DO NOTHING, 把幂等下沉到插入层。当前该入口无自动调用方, 窗口不可达。
2. 轮询实时性下限 **1.5s**: 体验已达标;WS 化是传输层替换 (§8)。
3. 代码冲突处理: 共享目录 + 单 Agent 串行 (MaxParallelPerAgent=1) 规避了大部分写冲突, 无自动合并。
4. **Codex** 适配器未在演示中启用: 代码就绪、已注册, 启用属配置 / 验证工作。
5. 工作区终端为房主信任模型: 不做命令白名单——与产品定位 (自己的开发台) 一致, 公网多租户部署前需加固。
6. 契约驱动 / 多级编排 / 安全监控等前沿设计为路线图 (见 §13 标 [路线图](#) 项与产品设计文档 §6 / §8)。